

Object Oriented Analysis and Design

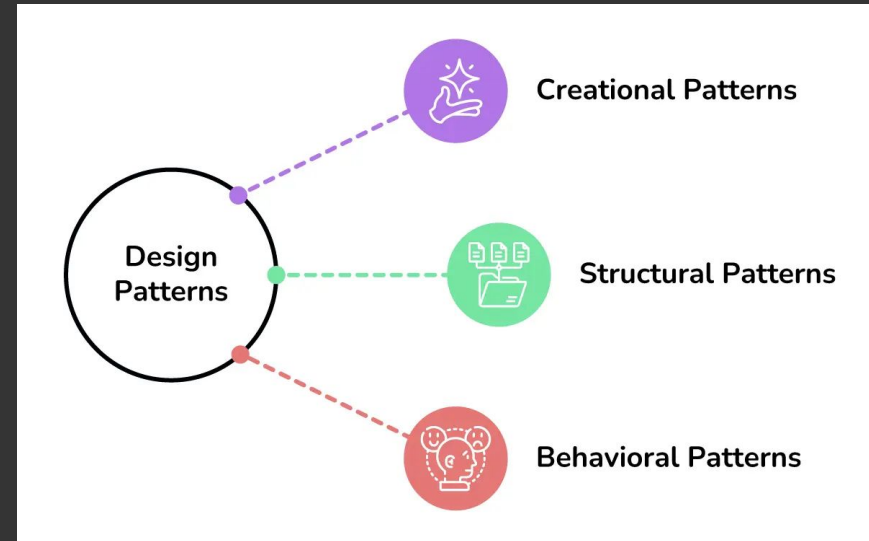
Design Patterns: Builder and Decorator

Vladimir Soroka



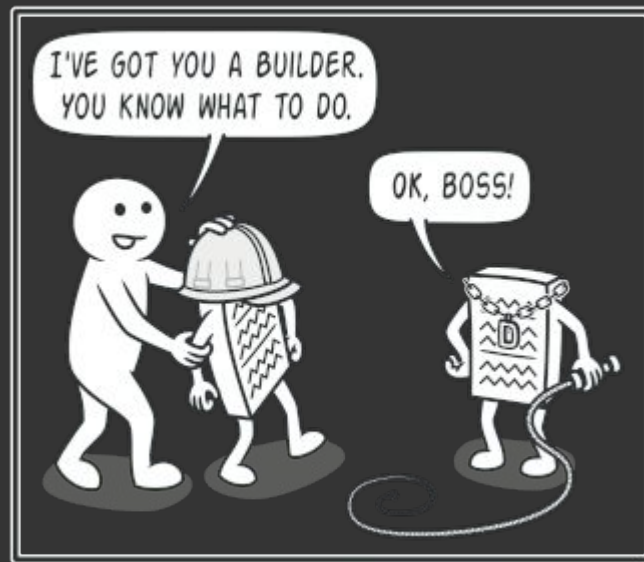
3 Categories Of Patterns

- **Creational** – object creation
- **Structural** – class/object composition
- **Behavioral** – object interaction & responsibility



What are we covering?

- **Builder** – creational
- **Decorator** - structural



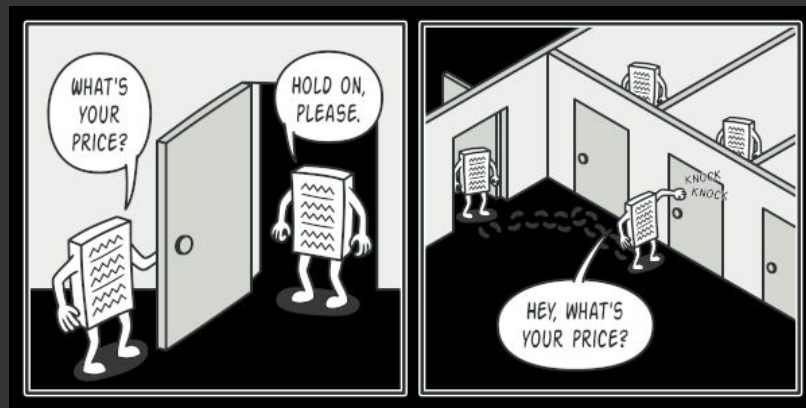
Reminder - The Composite Pattern

- **Motivation**

- You want to treat individual and groups of objects uniformly.

- **Use Case**

- File system, UI layout, organization chart



The Composite Code

```
class Graphic {
public:
    virtual void draw() = 0;
};

class Line : public Graphic {
public:
    void draw() override { std::cout << "Line\n"; }
};

class Picture : public Graphic {
    std::vector<Graphic*> children;
public:
    void add(Graphic* g) { children.push_back(g); }
    void draw() override {
        for (auto g : children) g->draw();
    }
};
```

The Builder Pattern

- **Purpose:** Separates the construction of a complex object from its representation so that the same construction process can create different representations.
- **Typical Use Case:** When you want to construct an object step-by-step, especially when it has many optional or configurable parts.
- **Example:**
 - Building a complex Meal object with optional drink, side, dessert.
 - Fluent interfaces like `.setName().setAge().setGender()` in modern languages.

```
House houseBuilder = new HouseBuilder()  
    .setFrontPorch(HouseBuilder.COLOR.WHITE,  
        HouseBuilder.SIZE.EXTRA_LARGE)  
    .build();
```



Builder UML

```
@startuml
skinparam classAttributeIconSize 0
```

```
' ==== Product ====
```

```
class Vehicle {
```

```
- wheels: int
```

```
- engine: String
```

```
- seats: int
```

```
+ setWheels(int): void
```

```
+ setEngine(String): void
```

```
+ setSeats(int): void
```

```
+ showSpecifications(): void
```

```
} ' ==== Builder ====
```

```
interface Builder {
```

```
+ createVehicle(): void
```

```
+ buildWheels(): void
```

```
+ buildEngine(): void
```

```
+ buildSeats(): void
```

```
+ getVehicle(): Vehicle
```

```
}
```

```
' ==== Concrete Builders ====
```

```
class CarBuilder {
```

```
- vehicle: Vehicle
```

```
+ createVehicle(): void
```

```
+ buildWheels(): void
```

```
+ buildEngine(): void
```

```
+ buildSeats(): void
```

```
+ getVehicle(): Vehicle
```

```
}
```

```
class BikeBuilder {
```

```
- vehicle: Vehicle
```

```
+ createVehicle(): void
```

```
+ buildWheels(): void
```

```
+ buildEngine(): void
```

```
+ buildSeats(): void
```

```
+ getVehicle(): Vehicle
```

```
}
```

```
' ==== Director ====
```

```
class Director {
```

```
- builder: Builder
```

```
+ setBuilder(Builder):  
void
```

```
+ construct(): Vehicle
```

```
}
```

```
' ==== Relationships ====
```

```
Builder <|.. CarBuilder
```

```
Builder <|.. BikeBuilder
```

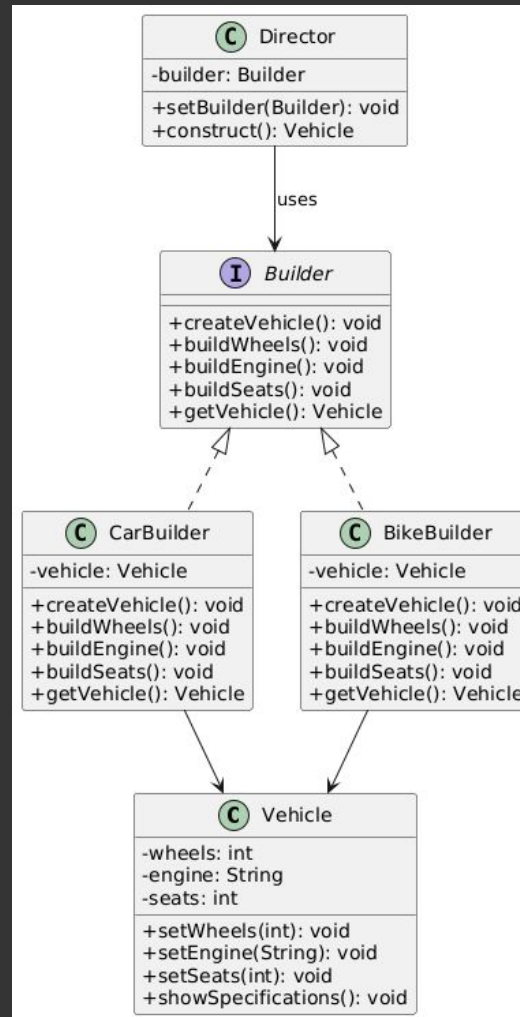
```
Director --> Builder :  
uses
```

```
CarBuilder --> Vehicle
```

```
BikeBuilder --> Vehicle
```

```
@enduml
```

Builder UML



Builder Code - 1

```
class Burger {
    string bun, patty, sauce;
public:
    void setBun(string b) { bun = b; }
    void setPatty(string p) { patty = p; }
    void setSauce(string s) { sauce = s; }
    void describe() {
        cout << "Burger with " << bun << ", " << patty << ", and " << sauce << endl;
    }
};

class BurgerBuilder {
protected:
    Burger* burger;
public:
    virtual void buildBun() = 0;
    virtual void buildPatty() = 0;
    virtual void buildSauce() = 0;
    Burger* getBurger() { return burger; }
    void createNewBurger() { burger = new Burger(); }
};
```

Builder Code - 2

```
class CheeseBurgerBuilder : public BurgerBuilder {
public:
    void buildBun() override { burger->setBun("Sesame"); }
    void buildPatty() override { burger->setPatty("Beef"); }
    void buildSauce() override { burger->setSauce("Cheddar"); }
};

class Director {
    BurgerBuilder* builder;
public:
    void setBuilder(BurgerBuilder* b) { builder = b; }
    Burger* construct() {
        builder->createNewBurger();
        builder->buildBun();
        builder->buildPatty();
        builder->buildSauce();
        return builder->getBurger();
    }
};
```

Builder Code - 3

```
class BurgerBuilder {
    Burger burger;

public:
    BurgerBuilder& addBun(const std::string& bun) {
        burger.setBun(bun);
        return *this;
    }

    BurgerBuilder& addPatty(const std::string& patty) {
        burger.setPatty(patty);
        return *this;
    }

    BurgerBuilder& addSauce(const std::string& sauce) {
        burger.setSauce(sauce);
        return *this;
    }

    Burger build() {
        return burger;
    }
};
```

What's Good About Builder?

1. Separation of Concerns

- Cleanly separates *how* a product is built from *what* it is.
- Result: Easier maintenance and testing.
- 🗨️ *“Let the Director worry about the sequence, and the Builder worry about the parts.”*



I want all
the component's
logic in one place



I prefer
locality of behavior
over
separation of concerns

2. Step-by-Step Construction

- Allows building objects gradually with fine-grained control over each step.
- Good for complex objects that can't be created in one constructor call.

What's Good About Builder?

3. Avoids Telescoping Constructors

- Replaces overly long constructor parameter lists:

```
new Pizza(cheese, crust, tomato,  
onions, olives, ...)
```

- With a clear and fluent alternative:

```
PizzaBuilder().withCheese().withOlives().build()
```



What's Good About Builder?

4. Encourages Immutability

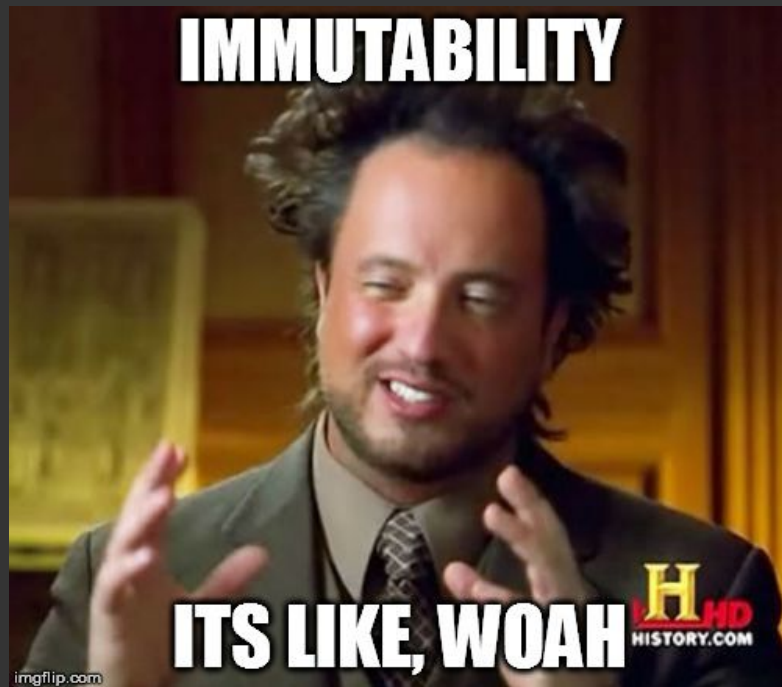
- You can build immutable objects by gathering all required values before construction.

5. Multiple Representations

- Same construction process can create different representations (e.g., a CarBuilder vs BikeBuilder).

6. Great for Fluent APIs

- Widely used in modern APIs (e.g., Java streams, query builders, UI DSLs) for readability and expressiveness.



What's Bad About Builder?

1. Code Overhead

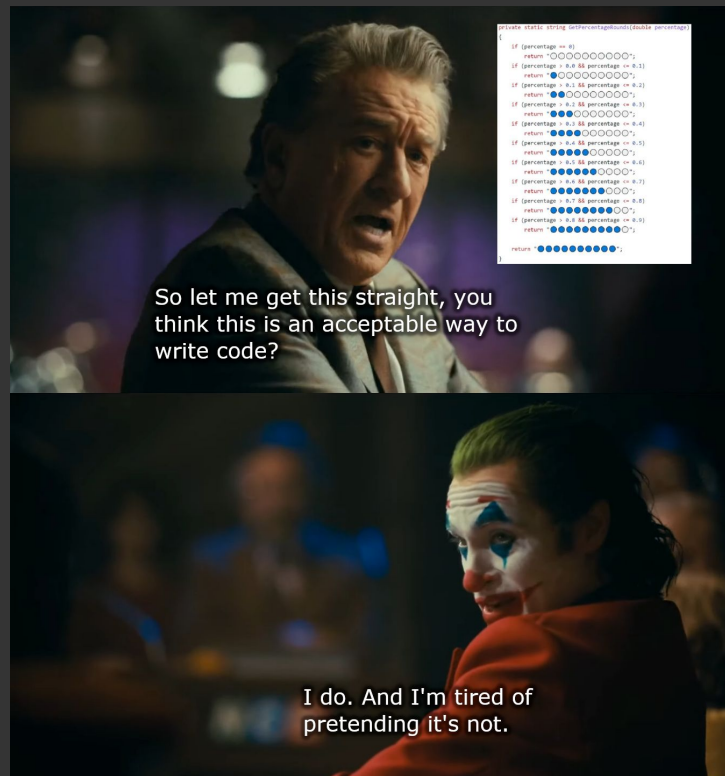
- You may end up writing many classes (Builder, ConcreteBuilder, Director, Product).
- **For simple objects**, this is over-engineering.

2. Misuse for Simple Objects

- Not every object needs a builder. Using one for trivial objects adds cognitive load.

3. More Classes, More Complexity

- Beginners may find it harder to trace how the object is actually created.



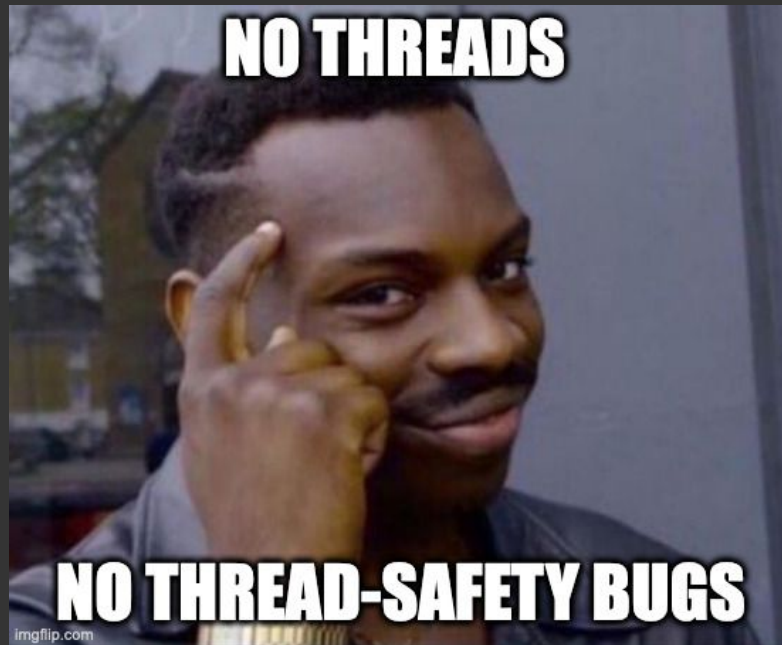
What's Bad About Builder?

4. Thread Safety Issues

- Stateful builders need care in multithreaded environments.
- `a().b().c().d()` is NOT atomic
- Builders often hold internal state that can be corrupted if reused across threads.

5. Inversion of Control is Limited

- Compared to Abstract Factory or Dependency Injection, Builder still requires the user to orchestrate the process or rely on a Director.



Builder Summary

When to Use It

Use Case	Use Builder?
Constructing a complex object step-by-step	✓ Yes
Need for different variations of a product	✓ Yes
Fluent, readable configuration	✓ Yes
Object has optional and required fields	✓ Yes
Simple object with just a few fields	✗ No

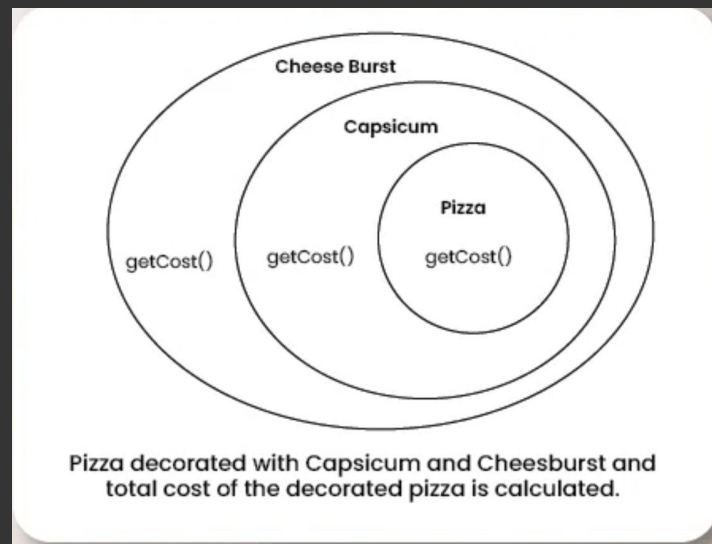
The Decorator Pattern

- **Problem**

- Adding new functionality often leads to subclass explosion:
 - TextViewWithBorder, TextViewWithScrollAndShadow, etc.
- Inflexible: All combinations need their own class
- Breaks Open/Closed Principle: Have to modify base class for changes

- **Solution**

- The **Decorator pattern** attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.



Real World Analogy

Coffee Order:

- Base: Espresso
- Add-ons: Milk, Whip, Vanilla

You don't need a separate class for EspressoWithMilkAndWhipAndVanilla.

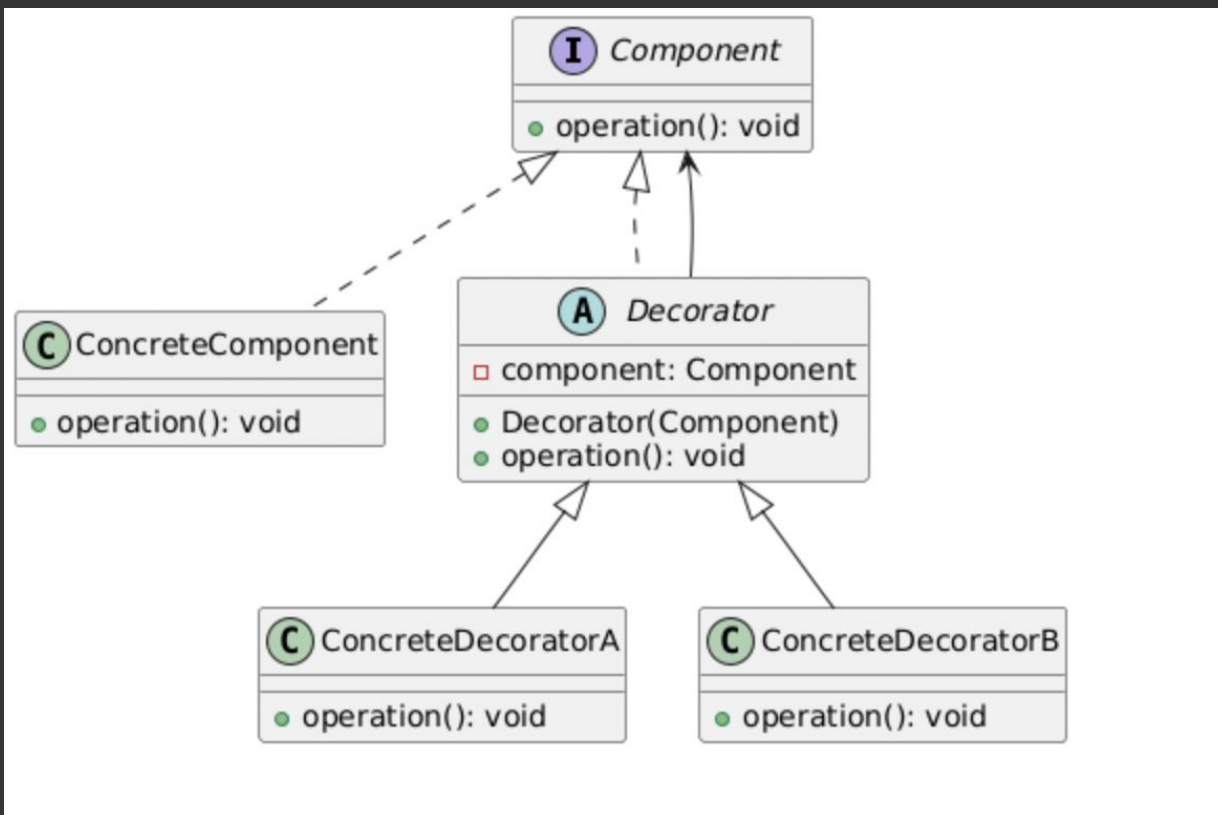
Each decorator wraps the base drink and adds its own cost and description.

Decorator UML

```
@startuml
interface Component {
    +operation(): void
}
class ConcreteComponent {
    +operation(): void
}
abstract class Decorator {
    -component: Component
    +Decorator(Component*)
    +operation(): void
}
```

```
class ConcreteDecoratorA {
    +operation(): void
}
class ConcreteDecoratorB {
    +operation(): void
}
Component <|.. ConcreteComponent
Component <|.. Decorator
Decorator <|-- ConcreteDecoratorA
Decorator <|-- ConcreteDecoratorB
Decorator --> Component
@enduml
```

Decorator UML



The Decorator Code

```
class TextDecorator : public Text {
protected:
    const Text* inner;
public:
    TextDecorator(const Text* t) : inner(t) {}
};

class Bold : public TextDecorator {
public:
    Bold(const Text* t) : TextDecorator(t) {}
    std::string render() const override {
        return "<b>" + inner->render() + "</b>";
    }
};

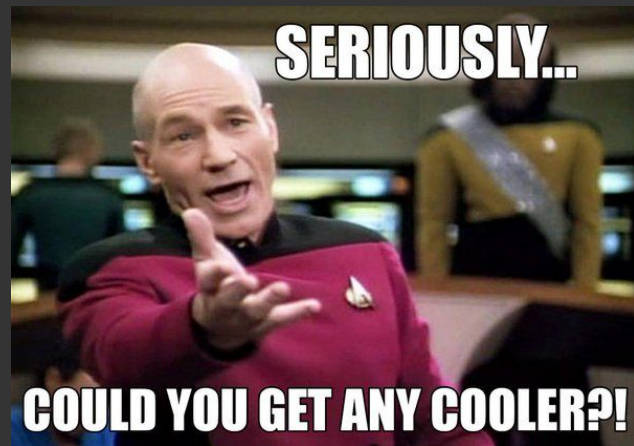
class Italic : public TextDecorator {
public:
    Italic(const Text* t) : TextDecorator(t) {}
    std::string render() const override {
        return "<i>" + inner->render() + "</i>";
    }
};
```

The Decorator Code

```
int main() {  
    Text* text = new Italic(new Bold(new PlainText()));  
    std::cout << text->render() << std::endl; // Output: <i><b>Hello</b></i>  
    delete text; // Don't forget to free memory  
}
```

Why Decorator Rocks?

- **✓ Open/Closed Principle**
Add new behavior without modifying existing code.
- **✓ Runtime Flexibility**
Stack behaviors dynamically, not just statically at compile-time.
- **✓ No Subclass Explosion**
No need to create ScrollBorderText, ScrollShadowText, etc.
- **✓ Fine-Grained Behavior**
Each decorator is focused and reusable (e.g., Bold, Italic, Underline).
- **✓ Same Interface**
Decorators are interchangeable with the base component (Component pattern).
- **✓ Used in Practice**
Java I/O, GUI frameworks, logging systems, middleware chains, and more.



Where Decorator Hurts?

- **✗ Too Many Small Classes**
One for each behavior = cluttered codebase if not well-organized.
- **✗ Debugging Complexity**
Behavior is layered: hard to trace which decorator added what.
- **✗ Order Matters**
`Bold(Italic(Plain))` $\neq$ `Italic(Bold(Plain))` — can lead to subtle bugs.
- **✗ Violated Expectations**
Users of the decorated object might not know all the behaviors it has.
- **✗ Violation of Liskov Substitution (LSP)**
If decorators change behavior *semantically*, not just additively.
- **✗ Memory Management in C++**
Ownership of wrapped objects needs to be handled carefully (use smart pointers!).



Bad Examples



Builder 1

```
class Point {  
    int x, y;  
public:  
    Point(int x, int y) : x(x), y(y) {}  
};
```

```
class PointBuilder {  
    int x = 0, y = 0;  
public:  
    PointBuilder& setX(int val) { x = val; return *this; }  
    PointBuilder& setY(int val) { y = val; return *this; }  
    Point build() { return Point(x, y); }  
};
```

Builder 2

```
UserBuilder builder;  
User u1 = builder.setName("Alice").setAge(30).build();  
User u2 = builder.setName("Bob").build();
```

Builder 3

```
builder.setName("John");  
registerWithSystem(builder.get());  
builder.setAge(40).build();
```

Builder 4

```
class UserBuilder {
    std::string email;
    std::string phone;

public:
    UserBuilder& setEmail(const std::string& e) {
        if (!isValidEmail(e) && !phone.empty())
            throw std::runtime_error("Invalid email when phone is already set");
        email = e;
        return *this;
    }

    UserBuilder& setPhone(const std::string& p) {
        phone = p;
        return *this;
    }

    User build() {
        return User(email, phone);
    }
};
```

```
builder.setEmail("bad@email").setPhone("123").build();
builder.setPhone("123").setEmail("bad@email").build();
```

Decorator 1

```
Stream* s = new FileStream();  
s->write(" Hello ");  
assert(s->getContent() == " Hello ");
```

```
Stream* s = new TrimmingStream(new FileStream());  
s->write(" Hello ");  
assert(s->getContent() == " Hello ");
```

Decorator 2

```
class Widget {  
public:  
    virtual void draw() const = 0;  
    virtual ~Widget() = default;  
};  
  
class Button : public Widget {  
public:  
    void draw() const override {  
        std::cout << "[ Button ]" << std::endl;  
    }  
};
```

```
class BorderDecorator : public Widget {  
    Widget* inner;  
    std::string borderColor = "black";  
public:  
    BorderDecorator(Widget* w) : inner(w) {}  
  
    void draw() const override {  
        std::cout << "Border (" << borderColor << ")\n";  
        inner->draw();  
    }  
  
    void setBorderColor(const std::string& color) {  
        borderColor = color;  
    }  
};
```


Decorator 3

```
class Text {
public:
    virtual std::string render() const = 0;
    virtual ~Text() = default;
};

class PlainText : public Text {
    std::string content;
public:
    PlainText(std::string t) : content(std::move(t)) {}
    std::string render() const override {
        return content;
    }
};
```

```
class AngryText : public TextDecorator {
public:
    using TextDecorator::TextDecorator;
    std::string render() const override {
        return "I AM ANGRY!!!";
    }
};
```

Decorator 4

```
class DataService {
public:
    virtual std::string fetchData() = 0;
    virtual ~DataService() = default;
};

class RemoteDataService : public DataService {
public:
    std::string fetchData() override {
        return "data-from-server";
    }
};
```

```
class CachingDataService : public DataService {
    DataService* inner;
    std::optional<std::string> cache;

public:
    CachingDataService(DataService* svc) : inner(svc) {}

    std::string fetchData() override {
        if (cache) return *cache;

        std::string data = inner->fetchData();
        cache = data;

        std::ofstream log("fetch.log", std::ios::app);
        log << "Fetched: " << data << std::endl;
        log.close();

        return data;
    }
};
```

